REPOSITORY DATA-FLOW REVIEW

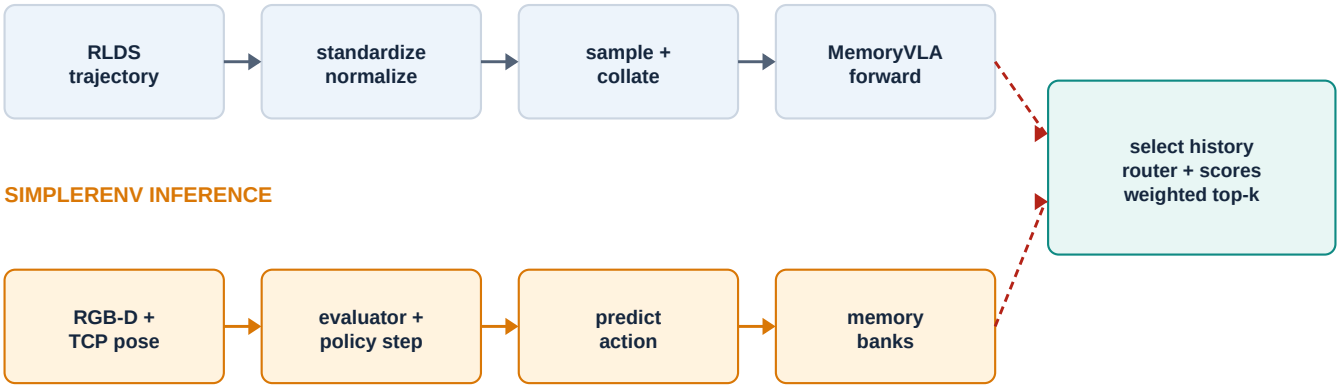
MemoryVLA Training and SimplerEnv Inference

A function-level trace of query-based retrieval metadata, memory-bank behavior, spatial processing, and action generation.

Primary finding Position and task type currently do not reach the weighted retrieval calculation. The standalone scorer is sound enough to pass its four focused scenarios, but integration fails earlier in both training and inference.

Scope	Repository	Review basis
Training pipeline; SimplerEnv inference; memory banks; query scoring	myMemoryVLA/ in the current workspace	Current working tree, including five uncommitted files; no files were modified during analysis

TRAINING



Executive summary

The retrieval total is a weighted sum of four scores. Position contributes to the spatial score; it does not populate the weights. Task type selects a preset weight profile and can also contribute to task matching when stored memories carry corresponding task tags.

```
total_score = semantic_weight * semantic_score
+ spatial_weight * spatial_score
+ temporal_weight * temporal_score
+ task_weight * task_score
```

Metadata	Where it should enter	Effect
Query embedding + memory embedding	RetrievalQuery.embedding; MemoryRecord.embedding	Semantic cosine similarity
Current + historical position	RetrievalQuery.current_position; MemoryRecord.position	Spatial distance score
Current + historical time	RetrievalQuery.current_time; MemoryRecord.timestamp	Temporal recency score
Task type + memory tags	RetrievalQuery.task_type; MemoryRecord.task_tags	Weight routing and task match
Object IDs + modality	Query and record metadata	Additional task relevance

Current outcome The integration is syntactically valid, but runtime-invalid. Undefined variables, mismatched keyword names, an incompatible memory-entry representation, and dropped metadata prevent spatial and task-aware retrieval from running.

Two different kinds of weights

- Retrieval weights in vla/spatial/retrieval.py combine semantic, spatial, temporal, and task relevance.
- Dataset sampling weights choose how often each RLDS dataset is sampled. They do not affect retrieval.
- Adaptive action-ensemble weights combine overlapping action predictions during SimplerEnv execution. They also do not affect retrieval.

1. Training data flow

The default path starts in train.py, builds a grouped episodic RLDS pipeline, collates PyTorch batches, and calls MemoryVLA.forward. The model derives cognition, perception, and spatial tokens before the diffusion action loss.

1.1 Training setup

File / function	Input	Output	Inside
train.py TrainConfig.__post_init__	Nested VLAConfig	Mutated TrainConfig	Copies epochs, batch sizes, optimizer settings, and strategy configuration to the top-level object.
train.py train	TrainConfig	None; performs training	Loads or constructs MemoryVLA, creates RLDS data and collator, configures FSDP, then starts the training loop. Retrieval mode and top-k reach MemoryVLA through vars(cfg).
vla/materialize.py get_vla_dataset_and_collator	Dataset mix, tokenizer, transforms, modality flags	Dataset, action tokenizer, collator	Creates RLDSBatchTransform and PaddedCollatorForActionPrediction. The default dataloader type creates GroupRLDSDataset.

Preexisting configuration constraint

Training defaults to use_spatial_features=True with a large OXE mixture. Most datasets in that mixture have no primary depth or intrinsics, and RLDSDataset.__init__ rejects them. The default all-spatial mixture is not valid without filtering or changing flags.

1.2 RLDS loading and standardization

File / function	Input	Output	Inside
oxe/materialize.py make_oxe_dataset_kwargs	Dataset name and modality flags	RLDS keyword dictionary	Reads observation mappings, state fields, intrinsics, transform, and normalization settings from OXE_DATASET_CONFIGS.
oxe/materialize.py get_oxe_dataset_kwargs_and_weights	Mixture specification	Per-dataset kwargs and sampling weights	Deduplicates datasets, materializes configs, and skips unsupported action encodings. Sampling weights are unrelated to retrieval weights.
oxe/transforms.py bridge_orig_dataset_transform	Raw Bridge trajectory	Standardized trajectory	Builds seven-dimensional actions, EEF_state, gripper state, and language instruction.
oxe/transforms.py maniskill_dataset_transform	Raw ManiSkill trajectory	Metric-depth trajectory	Converts fixed-point depth to meters and extracts gripper state.
rlds/dataset.py make_dataset_from_rlds	Per-dataset config	DLataset and statistics	Loads TFDS/RLDS, standardizes trajectories, normalizes action/proprio, and returns trajectory-level data.
rlds/dataset.py restructure (inner)	Standardized trajectory	Canonical trajectory dictionary	Renames RGB/depth fields, installs intrinsics, concatenates state into proprio, creates timestep, and moves language into task.language_instruction.
rlds/utils/data_utils.py normalize_action_and_proprio	Trajectory and statistics	Normalized trajectory	BOUNDS_Q99 maps action and proprio dimensions to approximately [-1, 1].
rlds/dataset.py apply_trajectory_transforms	Trajectory dataset	Chunked trajectory dataset	Filters unlabeled data, adds padding metadata, and invokes chunk_act_obs.
rlds/traj_transforms.py chunk_act_obs	Whole trajectory	Per-timestep windows	With window_size=1 and future window 15, each frame receives one observation and 16 actions.
rlds/obs_transforms.py decode_and_resize	Observation dictionary	Resized observation	Decodes RGB/depth, resizes both, and scales camera intrinsics consistently.
rlds/dataset.py make_interleaved_episodic_dataset	Multiple episodic datasets	Interleaved DLDataset	Loads and balances datasets, selects sorted random frame groups, and shuffles episode groups.
rlds/dataset.py group_sample (inner)	One trajectory	Exactly group_size frames	Pads short trajectories or randomly samples and then sorts timesteps, preserving order but not continuity.

1.3 Position and task type before batching

No explicit position field is created. Position is embedded in proprio, but it is not a safe universal retrieval coordinate:

- Proprio has already been normalized to approximately [-1, 1].
- For POS_EULER and POS_QUAT datasets, the first three values are EEF XYZ.
- For JOINT datasets, the first three values are joint angles.
- For state-less datasets, these values may be padding zeros.
- Robots and datasets use different coordinate frames.

Consequence	proprio[:, :3] is not a valid metric position for the full OXE mixture. A dedicated, unnormalized EEF-position field is needed for datasets whose coordinate semantics are known.

Only natural-language instruction exists as task information. No task_type field is produced by standardization, restructure, RLDSBatchTransform, the collator, or TrainingStrategy. The current configs.py edit is whitespace-only and has no data-flow effect.

1.4 PyTorch sample and batch creation

File / function	Input	Output	Inside
vla/datasets/datasets.py GroupRLDSDataset.make_dataset	RLDS config	Episodic DLDataset	Requests grouped episodic sampling.
vla/datasets/datasets.py GroupRLDSDataset.__iter__	Episode group	Individual frame dictionaries	Assigns a local episode ID and yields frames in the selected order.
vla/datasets/datasets.py RLDSBatchTransform.__call__	One NumPy RLDS frame	PyTorch example dictionary	Transforms RGB, tokenizes instruction, and converts actions, depth, proprio, and intrinsics to tensors.
prismatic/util/data_utils.py PaddedCollatorForActionPrediction.__call__	List of examples	Batched dictionary	Pads text; stacks pixels, actions, depth, proprio, and intrinsics; preserves images, instructions, episode IDs, and times.
training/strategies/base_strategy.py run_vla_training	Dataset, collator, metrics	None	Builds DataLoader and passes batch fields to MemoryVLA.forward under autocast before backpropagation and optimizer steps.

Relevant batch contract

```
pixel_values: [B, ...]
actions: [B, 16, 7]
depth: [B, H, W, 1]
intrinsics: [B, 3, 3]
proprio: [B, P]
timesteps: NumPy [B]
episode_ids: NumPy [B]
instructions: list[str], length B
images: list[PIL.Image], length B
```

Neither positions nor task_types is present in this batch.

1.5 Model forward and action loss

File / function	Input	Output	Inside
prismatic/models/vlms/prismatic.py PrismaticVLM.forward	Text IDs, mask, RGB pixels, labels	CausalLMOutputWithPast	Encodes RGB, saves raw patch features in self.vision_feats, projects image tokens, joins image and text, and runs the LLM.
vla/memory_vla.py _encode_retrieval_inputs	PIL images and instructions	Normalized CLIP image/text embeddings	Creates stored visual embeddings and current query embeddings.
vla/memory_vla.py MemoryVLA.forward	Full training batch	Scalar diffusion loss and VLM output	Extracts cognition/perception tokens, updates memory banks, creates a point cloud, fuses spatial memory, and computes diffusion loss.
vla/spatial/geometry.py depth_to_points	Depth [B,H,W], intrinsics [B,3,3]	Points [B,H*W,3], valid mask	Back-projects depth pixels into camera-frame XYZ.
vla/spatial/encoder.py PointCloudSpatialEncoder.forward	Points, optional context, validity mask	[B,num_spatial_tokens,256]	Encodes valid points and compresses them through learned-query cross-attention.
vla/memory_vla.py _fuse_spatial_tokens	Perception and spatial tokens	Fused perception tokens	Runs spatial memory, cross-attends perception to spatial history, and gate-fuses the result.
action_model/action_model.py ActionModel.loss	Action chunks, cognition, perception	Scalar MSE	Samples noise/time, creates noisy actions, predicts noise with DiT, and computes mean squared error.
action_model/models.py DiT.forward	Noisy actions, diffusion time, cognition, perception	Predicted noise [B,T,7]	Embeds conditions, applies Transformer blocks with perception attention, and returns noise predictions.

2. SimplerEnv inference data flow

The inference path merges checkpoint configuration with CLI arguments, loads MemoryVLA, constructs a simulator environment, and executes one policy step per observation. The position source added by the current work is the robot TCP pose.

2.1 Initialization and episode orchestration

File / function	Input	Output	Inside
evaluation/simpler_env/simpler_env_inference.py deep_update	YAML config and CLI dictionary	Merged dictionary	Recursively applies CLI overrides.
simpler_env_inference.py parse_local_args	CLI arguments	Local args and remaining args	Extracts BF16 and spatial-mode flags before SimplerEnv parses the rest.
simpler_env_inference.py main block	CLI and YAML	Model and results	Seeds libraries, constructs VLAInference, and calls maniskill2_evaluator.
vla/load.py load_vla	Checkpoint path and kwargs	MemoryVLA	Loads configuration, dataset statistics, vision/LLM backbones, and delegates checkpoint restoration.
vla/memory_vla.py MemoryVLA.from_pretrained	Checkpoint and backbones	Loaded MemoryVLA	Loads VLM, action model, memory modules, spatial encoder, and fusion weights.
evaluation/simpler_env/vla_policy.py VLAInference.__init__	Checkpoint and policy config	Stateful inference wrapper	Moves model to CUDA and configures histories, action ensemble, and robot-specific action conversion.
evaluation/simpler_env/maniskill2_evaluator.py maniskill2_evaluator	Model and benchmark arguments	list[bool]	Iterates robot/object initializations and invokes one episode at a time.

2.2 Per-episode and per-step flow

File / function	Input	Output	Inside
maniskill2_evaluator.py run_maniskill2_eval_single_episode	Model, env config, initial poses	Boolean success	Builds and resets env, extracts RGB-D, loops policy steps, applies actions, and writes probe/video outputs.
SimplerEnv observation_utils.py get_image_depth_intrinsics_from_maniskill2_obs_dict	Env and observation	RGB plus depth/intrinsic/extrinsic tensors	Selects the robot camera and extracts calibrated RGB-D data.
vla_policy.py VLAInference.reset	Task description	None	Clears wrapper histories, ensemble, and gripper state. It does not directly reset model memory banks.
vla_policy.py _add_*_to_history	Current RGB/depth/intrinsic	None	Adds sensor values to wrapper deques. These deques are separate from model memory and are not passed into MemoryVLA.
vla_policy.py VLAInference.step	RGB, depth, intrinsics, instruction, current position	Raw and environment action dictionaries	Calls predict_action, ensembles actions, converts rotation to axis-angle, and maps the gripper command.
vla/memory_vla.py MemoryVLA.predict_action	Image, instruction, depth/intrinsics, current position	Unnormalized and normalized [16,7] actions	Runs VLM generation, updates memory, creates spatial tokens, samples actions, and unnormalizes them.
vla/memory_vla.py _add_batch_depth	Unbatched depth	Batched model-device depth	Normalizes channel/batch axes and moves to spatial-encoder device/dtype.
vla/memory_vla.py _add_batch_intrinsics	[3,3] intrinsics	[1,3,3] intrinsics	Adds batch dimension and moves to model device/dtype.
adaptive_ensemble.py ensemble_action	Current predicted action chunk	One seven-dimensional action	Aligns overlapping predictions and averages them with cosine-similarity weights. These are not retrieval weights.

Intended SimplerEnv position

np.asarray(base_env.tcp.pose.p, dtype=np.float32) is the current end-effector position in the simulator world frame. This is a reasonable runtime position source once base_env is initialized before the first policy step.

3. Shared memory-bank and retrieval flow

All three banks - cognition, perception, and spatial - inherit CogMemBank behavior. Each bank uses the same episode/time/instruction mechanics but stores features with a different token dimension.

3.1 Memory-bank functions

File / function	Input	Output	Inside
vla/memory_vla.py CogMemBank.reset	None	None	Clears every episodic memory and the stream episode marker.
CogMemBank.clear_episode	Episode ID	None	Removes one episode's history.
CogMemBank._memory_consolidate	Episode, feature, time, embedding, proposed position/tags	None	Appends a record and enforces memory length with FIFO or token merging.
CogMemBank._consolidate_with_token_merge	Episode ID	None	Finds the most similar adjacent features, averages them, and removes one entry.
CogMemBank.process_batch	Tokens plus episode/time/instruction/retrieval metadata	Context-enhanced tokens	Handles boundaries, selects history, cross-attends current tokens to history, fuses them, and stores the current memory.
CogMemBank._select_history	History, current tokens, instruction, time, embedding	Selected history list	Returns all history in off mode, random top-k in shuffled mode, or builds query/records for weighted retrieval.
CogMemBank._as_float_timestep	Tensor, number, or None	Float or None	Converts time into retrieval metadata.
CrossTransformerBlock.forward	Query, keys, values	Context-enhanced query	Performs cross-attention, residual normalization, and feed-forward processing.
GateFusion.forward	Current and retrieved features	Fused feature	Learns a per-element interpolation gate.

3.2 Retrieval functions

File / function	Input	Output	Inside
vla/spatial/retrieval.py ManualRetrievalRouter.route	RetrievalQuery	Mode and RetrievalWeights	Uses a classifier, explicit task type, or text heuristics.
ManualRetrievalRouter._mode_from_task_type	Query task type	Mode or None	Looks up aliases including navigation, object_state, and recent.
ManualRetrievalRouter._mode_from_text	Instruction text	Mode	Searches for navigation, recency, and state keywords.
semantic_score	Query and record embeddings	Float [0,1]	Computes cosine similarity and clamps negative values to zero.
spatial_score	Current and historical position	Float [0,1]	Computes exp(-distance / spatial_scale); missing position produces zero.
temporal_score	Current and historical time	Float [0,1]	Computes exp(-age / temporal_scale).
task_score	Query type/objects/modalities and memory tags	Float [0,1]	Adds 0.5 for task-type match, 0.3 for object match, and 0.2 for modality match.
MemoryRetriever.retrieve	Query, records, top-k	Sorted RetrievalResult list	Routes weights, calculates four component scores, computes totals, sorts descending, and truncates.

4. Exact integration failures

The following failures are ordered roughly from earliest runtime failure to deeper semantic problems.

#	Failure	Effect
1	Inference reads base_env before assignment	The first call to model.step reads base_env.tcp.pose.p, but base_env = env.unwrapped occurs only after env.step.
2	predict_action reads cog_tokens before assignment	positions is moved to cog_tokens.device before VLM generation creates cog_tokens. This raises UnboundLocalError.
3	Training uses the wrong keyword	CogMemBank.process_batch accepts positions, while MemoryVLA.forward passes current_position. This raises an unexpected-keyword TypeError.
4	task_type is undefined	MemoryVLA.forward neither accepts nor defines task_type, but passes it to cognition and perception banks.
5	current_position can be undefined	When proprio is absent, the else branch sets proprio=None instead of current_position=None.
6	process_batch reads instruction too early	The router call occurs before the per-sample loop that defines instruction.
7	BankEntry breaks tuple consumers	History was a three-tuple. It is now a five-field dataclass, but consolidation, retrieval, and feature extraction still destructure three-tuples.
8	Detached embedding is not stored	_memory_consolidate creates stored_embedding but stores the original image_embedding.
9	Position and tags never reach consolidation	Both calls to _memory_consolidate omit position and task_tags.
10	Selection never receives metadata	_select_history has no position or task-type parameters.
11	Query and records omit metadata	RetrievalQuery.current_position/task_type and MemoryRecord.position/task_tags are never populated.
12	Spatial fusion drops positions	_fuse_spatial_tokens accepts positions but does not pass them into spatial_mem_bank.process_batch.
13	Token merging drops new fields	The merge function writes an old-style three-tuple, losing position and task tags and creating a mixed-format bank.
14	Training has no task-type producer	Task type must be derived per instruction or explicitly carried through dataset, transform, collator, strategy, and model.
15	Training coordinates are not comparable	proprio[:, :3] is normalized and may contain joint angles or padding rather than metric EEF XYZ.

5. Additional conceptual issues

Issue	Why it matters
Overbroad text routing	STATE_TERMS contains the bare substring 'on'. Because matching uses term in text, ordinary instructions such as 'put carrot on plate' can route to OBJECT_STATE. Even words containing those letters can match.
Constant task contribution	Within an ordinary single-task episode, all memories may share the same task tag and modality. task_score then adds nearly the same constant to every candidate and does not materially change ranking.
Weak tabletop spatial discrimination	With spatial_scale=2.0, a 10 cm distance gives exp(-0.1/2), approximately 0.95. Position differences inside a tabletop workspace may therefore look almost identical.
Retrieval starts after top-k	If history length is less than or equal to top-k, _select_history returns all history without scoring. With top-k four, weighted selection begins only at five stored records.
Grouped memory lifetime	Grouped training clears the bank at the start of each batch. Retrieval sees only the current sampled episode group rather than long-lived history across batches.
Current position semantics	The inference source is end-effector position, not object or target position. That may be correct, but it should be an explicit design choice because spatial relevance will mean proximity of robot poses.

Standalone validation

All four focused retrieval scenarios pass when called directly: last-seen semantic/spatial/recent matching, object-state matching, audio-temporal-visual matching, and navigation weighting. This confirms that the scorer itself works independently of MemoryVLA integration.

6. Recommended metadata contract

Use one canonical, batched contract for every memory bank:

```
positions: Optional[torch.Tensor] # shape [B, 3], metric, common frame
task_types: Optional[list[str | None]] # length B
```

Inside process_batch, resolve each sample independently:

```
instruction_i = instructions[i]
position_i = positions[i] if positions is not None else None

task_type_i = (
task_types[i]
if task_types is not None
else router.route(RetrievalQuery(text=instruction_i))[0].value
)
```

Then carry the values through both sides of retrieval:

- Set RetrievalQuery.current_position to position_i.
- Set RetrievalQuery.task_type to task_type_i.
- Store BankEntry.position as a detached clone of position_i.
- Store BankEntry.task_tags as (task_type_i,).
- Convert every BankEntry into a MemoryRecord with position and task_tags.
- Make token merging preserve all fields instead of reverting to a tuple.
- Pass identical metadata to cognition, perception, and spatial memory banks.

6.1 Training-specific requirement

Create an explicit unnormalized EEF-position field before proprio normalization. Only enable it for datasets whose state encoding and coordinate frame are known. Do not infer it from the first three proprio values across a heterogeneous OXE mixture.

6.2 SimplerEnv-specific requirement

Initialize base_env = env.unwrapped immediately after env.reset, then pass the metric TCP pose on every step. Construct the position tensor after a valid model device is available, and forward it into every memory bank and record.

6.3 Suggested implementation order

1. Restore a single BankEntry representation and update every consumer.

2. Fix inference ordering: base_env, then model step; generate cognition tokens before using their device.

3. Standardize process_batch parameter names and per-sample metadata shapes.

4. Populate RetrievalQuery and MemoryRecord fields.

5. Add explicit training position/task metadata only after defining dataset semantics.

6. Add integration tests that prove spatial_score and task_score are nonzero inside MemoryVLA, not just in standalone retrieval tests.

Bottom line The retrieval mathematics is not the confusing part. The real issue is the metadata contract: source semantics, batched shape, per-sample routing, storage representation, and preservation through consolidation must agree across training and inference.

Appendix A. End-to-end field trace

Stage	Position	Task information	Current status
Training raw RLDS	Usually embedded in raw state	Language instruction	No explicit common contract
Training standardization	Concatenated into proprio	Language moved to task dict	Position semantics vary by dataset
Training normalization	Normalized with dataset statistics	Unchanged language	Metric scale is lost
RLDSBatchTransform	proprio tensor	instruction string	No position/task_type fields
Collator	batched proprio	list of instructions	No position/task_type fields
MemoryVLA.forward	Attempts proprio[:, :3]	Undefined task_type	Runtime broken
SimplerEnv evaluator	TCP pose intended	Environment instruction	base_env used before assignment
VLAInference.step	Passes current_position	Passes task description	Flow reaches predict_action
predict_action	Attempts positions tensor	Instruction available	cog_tokens device used too early
process_batch	positions parameter exists	task_type parameter exists	Instruction used before definition; values not used
_select_history	Not accepted	Not accepted	Query omits both fields
MemoryRecord	Field exists	task_tags field exists	Neither is populated by integration
MemoryRetriever	Scorer works	Router/scorer work	Receives zeros/missing metadata in integration

Appendix B. Review notes

- The current working tree contained uncommitted edits in maniskill2_evaluator.py, vla_policy.py, configs.py, memory_vla.py, and retrieval.py. They were inspected but not changed.
- The edited Python files parse successfully as Python syntax.
- The standalone retrieval test functions were invoked directly and all four passed.
- A full MemoryVLA runtime integration test was not available in the system Python because the timm dependency was missing. The integration failures above are direct control/data-flow errors visible before model execution.

End of report